FIFA 21 PLAYER MARKET & PERFORMANCE ANALYSIS

DATA VISUALIZATION REPORT -

1. The "Age Cliff" (Financial & Performance)

Goal: Visualize the peak performance vs. market value decline.

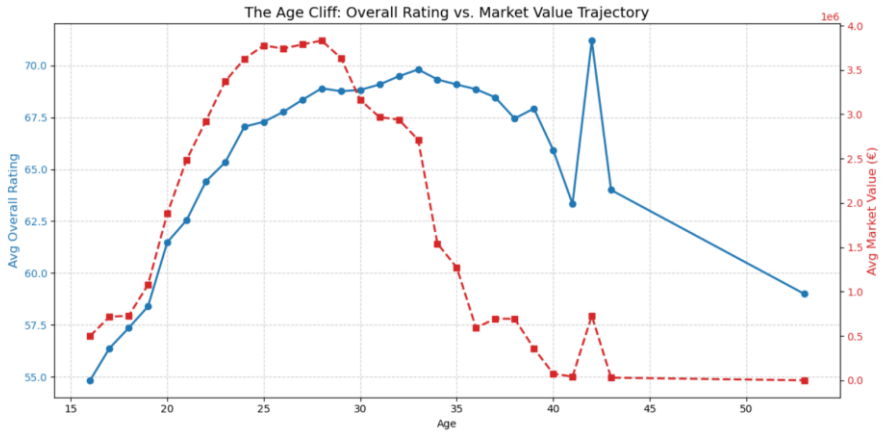
Plot: A dual-axis line chart.

- **X-axis:** Age
- **Primary Y-axis:** Average Overall
- **Secondary Y-axis:** Average Market Value

SOLUTION:

In [20]:
```
import matplotlib.pyplot as plt

# 1. Prepare data: Group by Age
age_stats = df.groupby('Age').agg({'Overall': 'mean', 'Value': 'mean'})

# 2. Setup the plot
fig, ax1 = plt.subplots(figsize=(12, 6))

# 3. Plot Overall Rating (Primary Y-axis)
color = 'tab:blue'
ax1.set_xlabel('Age')
ax1.set_ylabel('Avg Overall Rating', color=color, fontsize=12)
ax1.plot(age_stats.index, age_stats['Overall'], color=color, marker='o', linewidth=2, label='Overall Rating')
ax1.tick_params(axis='y', labelcolor=color)
ax1.grid(True, linestyle='--', alpha=0.6)

# 4. Create the second Y-axis for Market Value
ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Avg Market Value (€)', color=color, fontsize=12)
ax2.plot(age_stats.index, age_stats['Value'], color=color, marker='s', linestyle='--', linewidth=2, label='Market Value')
ax2.tick_params(axis='y', labelcolor=color)

# 5. Final touches
plt.title('The Age Cliff: Overall Rating vs. Market Value Trajectory', fontsize=14)
fig.tight_layout()
plt.show()
```

OUTPUT:



2. The "Hype Tax" (Hits vs. Value)

Goal: Prove that popularity inflates player market value.

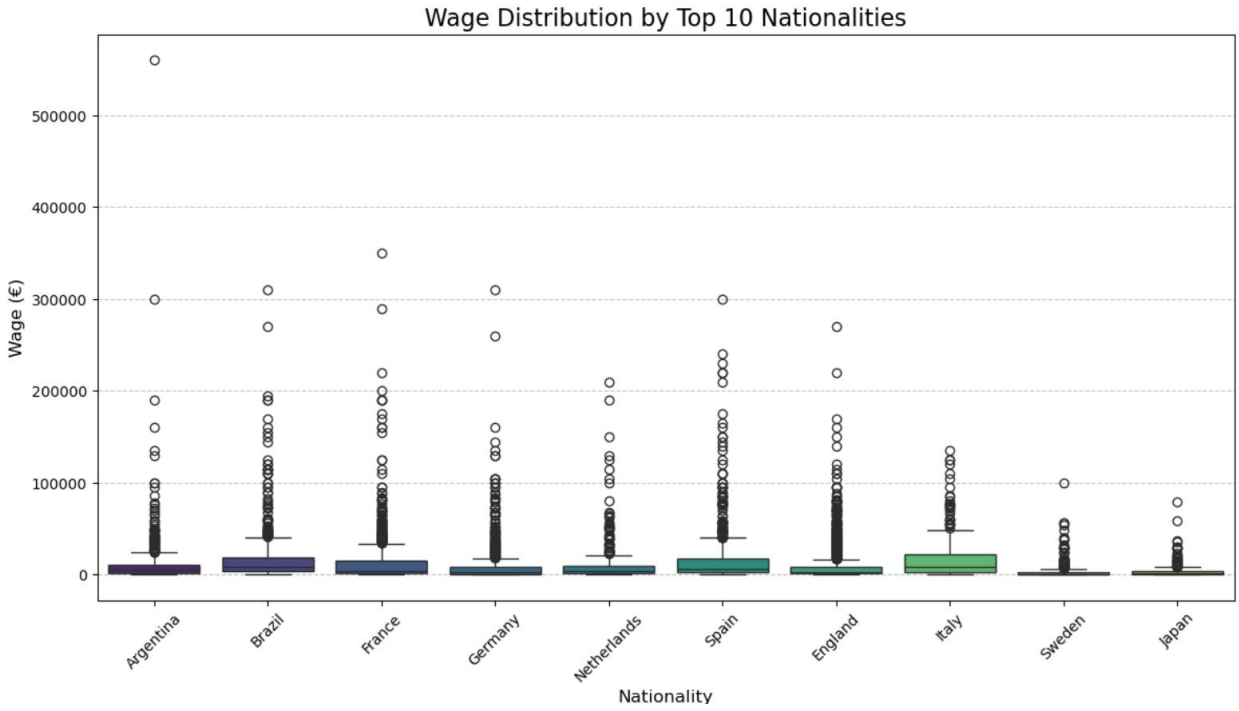
Plot: Scatter plot with a regression trendline.

- **X-axis:** Hits
- **Y-axis:** Market Value

SOLUTION:

```
In [21]: import seaborn as sns
         import matplotlib.pyplot as plt

         # 1. Identify the top 10 nationalities by count
         top_10_nationalities = df['Nationality'].value_counts().nlargest(10).index

         # 2. Filter the dataframe for these top 10
         df_top_10 = df[df['Nationality'].isin(top_10_nationalities)]

         # 3. Create the Box Plot
         plt.figure(figsize=(14, 7))
         sns.boxplot(data=df_top_10, x='Nationality', y='Wage', palette='viridis')

         # 4. Final touches
         plt.title('Wage Distribution by Top 10 Nationalities', fontsize=16)
         plt.xlabel('Nationality', fontsize=12)
         plt.ylabel('Wage (€)', fontsize=12)
         plt.xticks(rotation=45)
         plt.grid(axis='y', linestyle='--', alpha=0.7)

         plt.show()
```

OUTPUT:



3. The "Versatility Premium"

Goal: Show how player versatility (playing multiple positions) affects average wage.

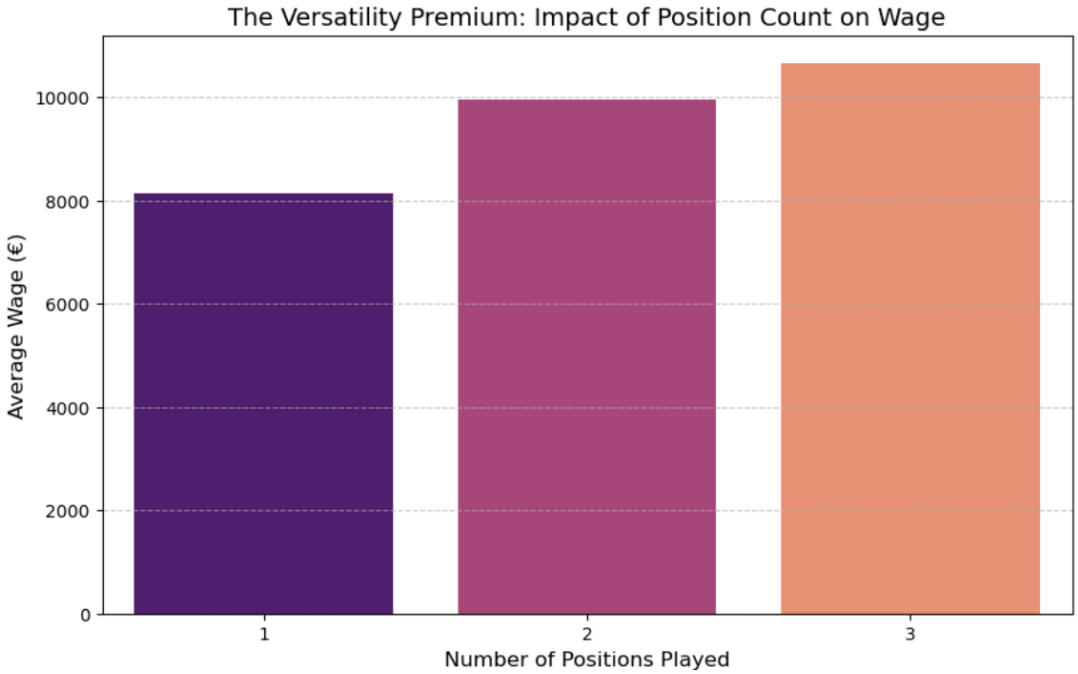
Plot: Grouped bar chart.

- **X-axis:** Number of Positions (1, 2, 3+)
- **Y-axis:** Average Wage

SOLUTION:

In [22]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Ensure Num_Positions is calculated (if not already done)
# df['Num_Positions'] = df['Positions'].apply(lambda x: len(x.split(',')))

# 2. Group data for the chart
versatility_data = df.groupby('Num_Positions')['Wage'].mean().reset_index()

# 3. Create the Bar Chart
plt.figure(figsize=(10, 6))
sns.barplot(data=versatility_data, x='Num_Positions', y='Wage', palette='magma')

# 4. Final touches
plt.title('The Versatility Premium: Impact of Position Count on Wage', fontsize=14)
plt.xlabel('Number of Positions Played', fontsize=12)
plt.ylabel('Average Wage (€)', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()
```

OUTPUT:



4. Financial Efficiency Leaders

Goal: Identify the smartest clubs by comparing their financial efficiency.

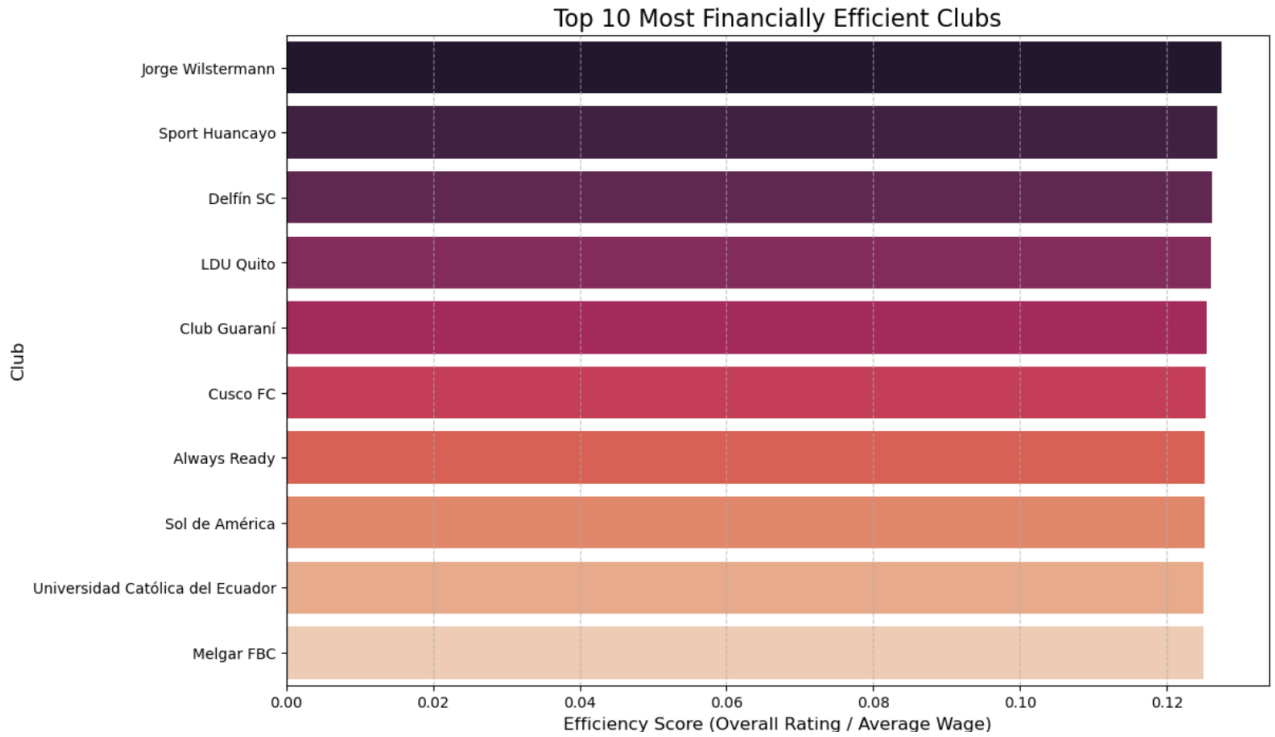
Plot: Horizontal bar chart of the Top 10 most efficient clubs.

- **X-axis:** Efficiency Score
- **Y-axis:** Club Name

SOLUTION:

In [24]:
```
# 1. Ensure the calculation happens first
# Make sure this runs before sorting
club_stats['Efficiency_Score'] = club_stats['Overall'] / club_stats['Wage']

# 2. Now filter out 'No Club'
club_stats = club_stats[club_stats.index != 'No Club']

# 3. Now sort by the score
club_stats = club_stats.sort_values(by='Efficiency_Score', ascending=False)

# 4. Select top 10
top_10_efficient = club_stats.head(10)

# 5. Create the Horizontal Bar Chart
plt.figure(figsize=(12, 8))
sns.barplot(data=top_10_efficient, x='Efficiency_Score', y=top_10_efficient.index, palette='rocket')

plt.title('Top 10 Most Financially Efficient Clubs', fontsize=16)
plt.xlabel('Efficiency Score (Overall Rating / Average Wage)', fontsize=12)
plt.ylabel('Club', fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()
```

OUTPUT:



5. Physicality vs Overall (The "Myth-Buster")

Goal: Investigate whether greater height or weight leads to a higher overall rating.

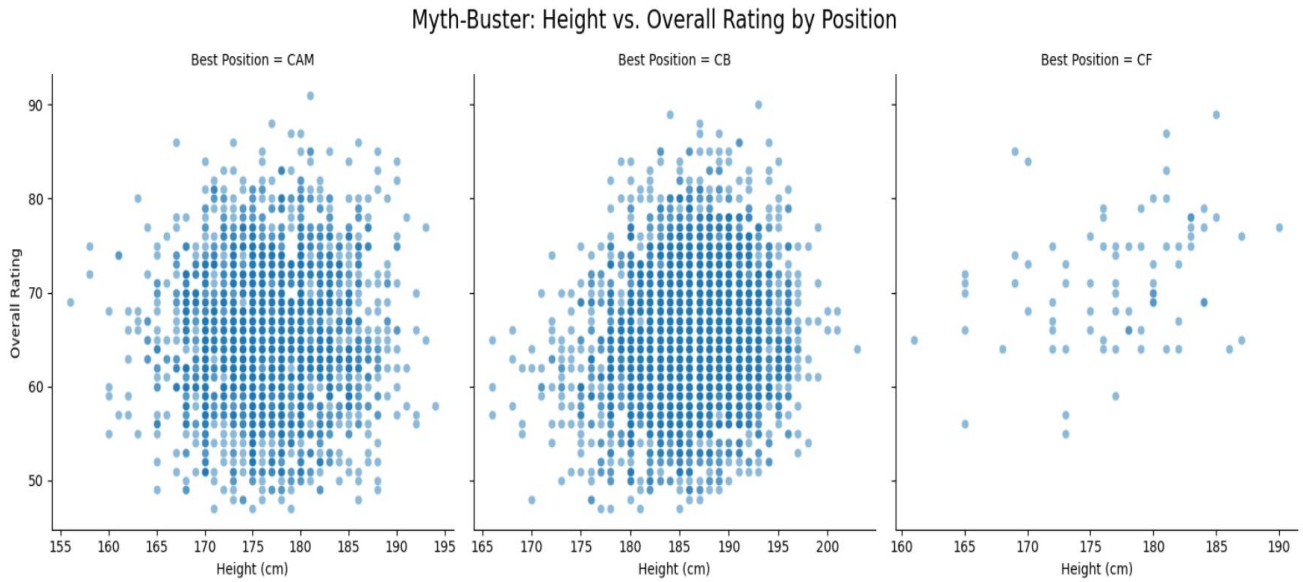
Plot: Heatmap or a grid of scatter plots for different positions (CF, CB, CAM).

- **X-axis:** Height / Weight
- **Y-axis:** Overall Rating

SOLUTION:

In [25]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Filter for specific positions to keep the chart clean
# Focusing on positions where physical presence is often debated
positions_to_plot = ['CF', 'CB', 'CAM']
df_phys = df[df['Best Position'].isin(positions_to_plot)]

# 2. Use FacetGrid to create a grid of plots
g = sns.FacetGrid(df_phys, col="Best Position", height=5, sharex=False, sharey=True)

# 3. Map the scatter plot to the grid
# Using Height on X-axis (ensure your dataset has a 'Height' column)
g.map(sns.scatterplot, "Height", "Overall", alpha=0.5)

# 4. Final touches
g.set_axis_labels("Height (cm)", "Overall Rating")
g.fig.suptitle('Myth-Buster: Height vs. Overall Rating by Position', y=1.05, fontsize=16)
plt.show()
```

OUTPUT:



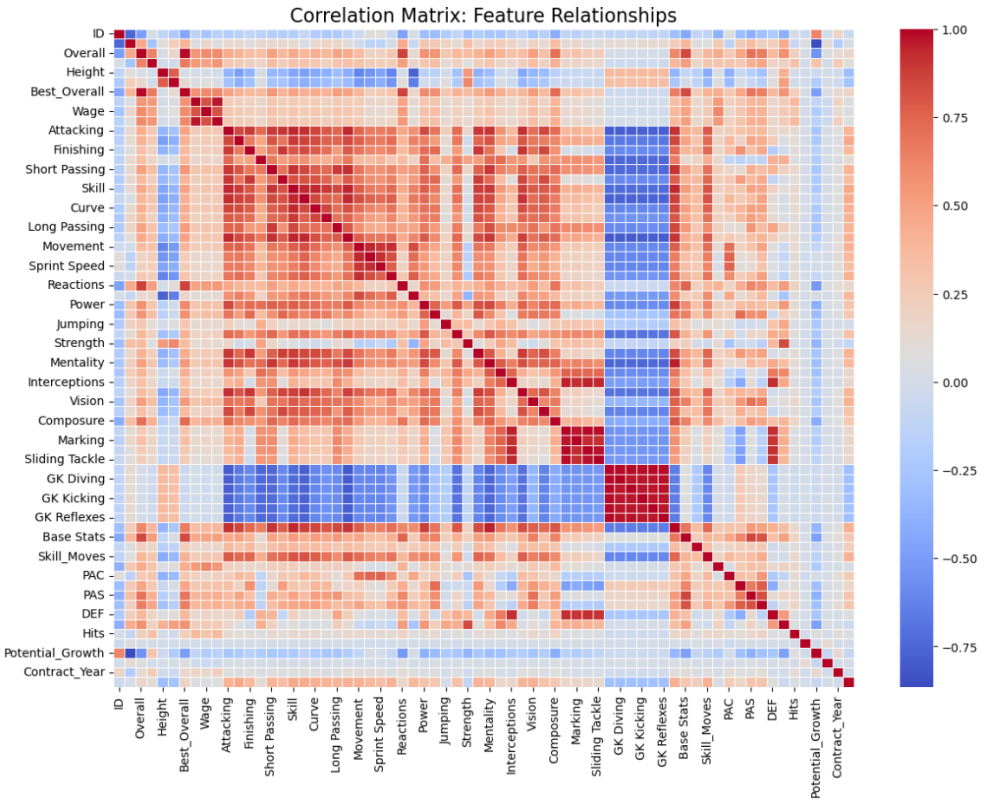
6. The Correlation Heatmap (Feature Relationship)

Goal: Visualize the relationships and correlations among all numerical features in the dataset.

SOLUTION:

In [30]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Select only numerical columns for correlation
numerical_df = df.select_dtypes(include=['float64', 'int64'])

# 2. Compute the correlation matrix
corr_matrix = numerical_df.corr()

# 3. Create the Heatmap
plt.figure(figsize=(14, 10))
sns.heatmap(corr_matrix, annot=False, cmap='coolwarm', linewidths=0.5)

# 4. Final touches
plt.title('Correlation Matrix: Feature Relationships', fontsize=16)
plt.show()
```

OUTPUT:

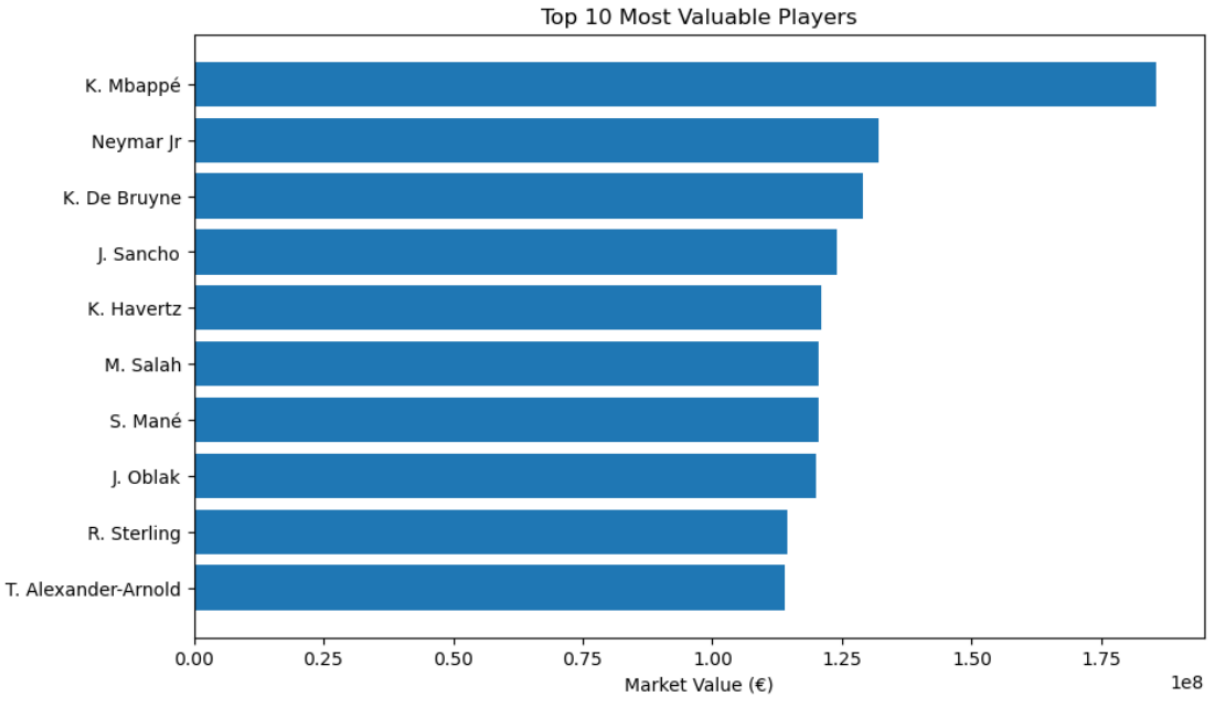


7. Top 10 Most Valuable Players

SOLUTION:

```
In [34]: top10 = df.nlargest(10,'Value')

plt.figure(figsize=(10,6))

plt.barh(top10['Name'], top10['Value'])

plt.title("Top 10 Most Valuable Players")

plt.xlabel("Market Value (€)")

plt.gca().invert_yaxis()

plt.show()
```

OUTPUT:

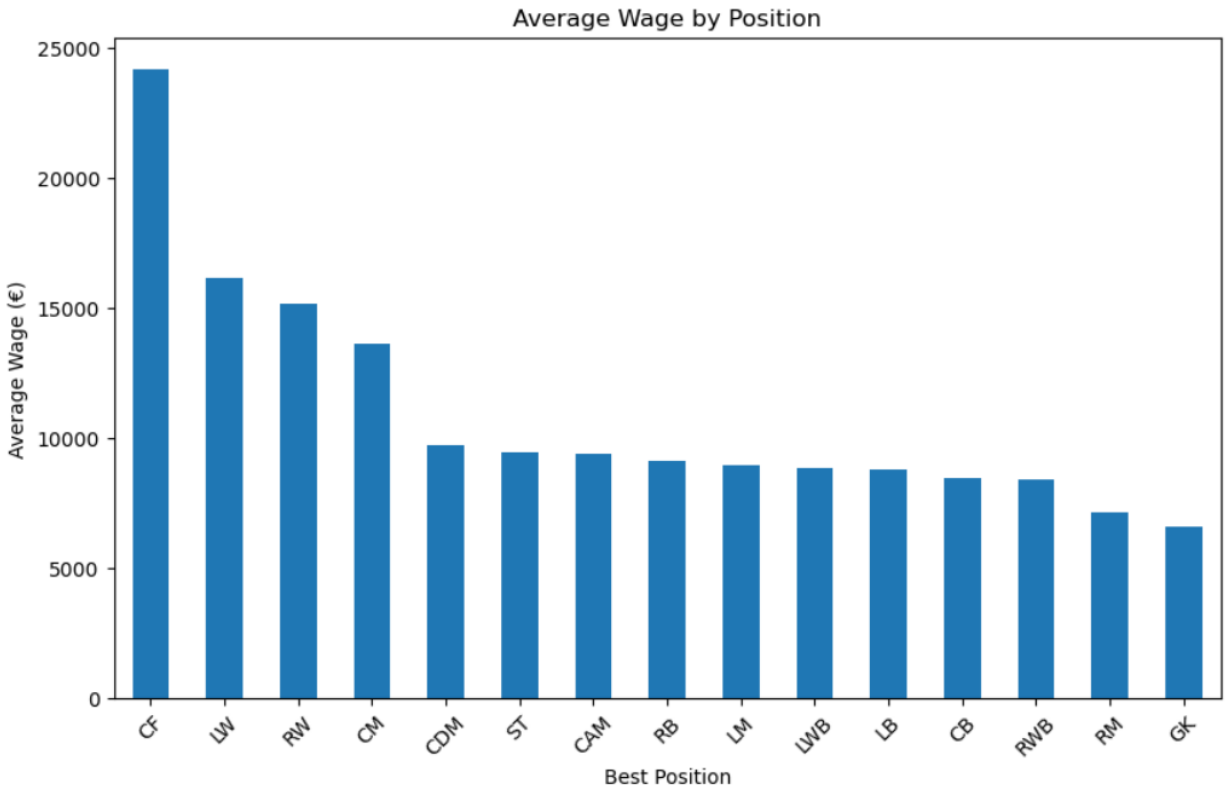


8. Average Wage by Position

SOLUTION:

```
In [35]: avg_wage = df.groupby('Best Position')['Wage'].mean().sort_values(ascending=False)

         plt.figure(figsize=(10,6))

         avg_wage.plot(kind='bar')

         plt.title("Average Wage by Position")

         plt.ylabel("Average Wage (€)")

         plt.xticks(rotation=45)

         plt.show()
```

OUTPUT:



9. Nationality Performance Map (Geographic Density)

Goal: Identify which countries produce the highest number of players and compare their average overall ratings to highlight global football talent hubs.

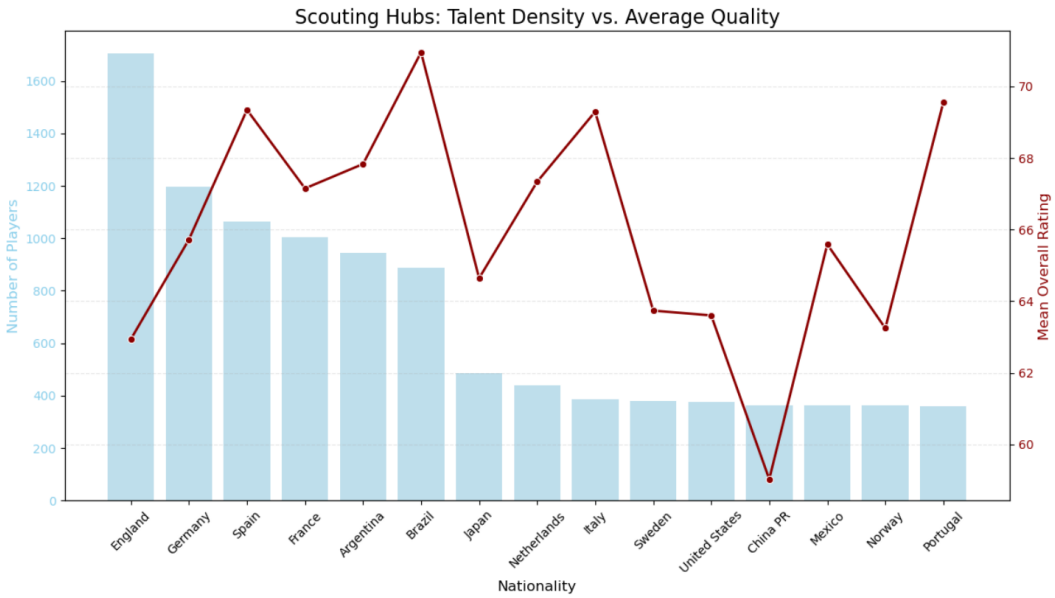
Plot: World map (using Plotly) or a horizontal bar chart of the Top 15 nationalities.

- **X-axis:** Nationality (Top 15 by player count)
- **Y-axis:** Mean Overall Rating

SOLUTION:

In [32]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Calculate Average Overall and Count per Nationality
nationality_stats = df.groupby('Nationality').agg({'Overall': 'mean', 'Name': 'count'})
nationality_stats = nationality_stats.rename(columns={'Name': 'Player_Count'})

# 2. Filter for Top 15 Nationalities by player count
top_15_nations = nationality_stats.nlargest(15, 'Player_Count')

# 3. Create a dual-axis plot: Bar for count, Line for mean rating
fig, ax1 = plt.subplots(figsize=(14, 7))

# Bar chart for Player Count
color1 = 'skyblue'
sns.barplot(x=top_15_nations.index, y=top_15_nations['Player_Count'], ax=ax1, color=color1, alpha=0.6)
ax1.set_ylabel('Number of Players', fontsize=12, color=color1)
ax1.tick_params(axis='y', labelcolor=color1)
ax1.set_xlabel('Nationality', fontsize=12)
ax1.set_xticklabels(top_15_nations.index, rotation=45)

# Line chart for Mean Overall Rating
ax2 = ax1.twinx()
color2 = 'darkred'
sns.lineplot(x=top_15_nations.index, y=top_15_nations['Overall'], ax=ax2, color=color2, marker='o', linewidth=2)
ax2.set_ylabel('Mean Overall Rating', fontsize=12, color=color2)
ax2.tick_params(axis='y', labelcolor=color2)

plt.title('Scouting Hubs: Talent Density vs. Average Quality', fontsize=16)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.show()
```

OUTPUT:



10. The "Age Distribution" Histogram

Goal: Visualize the age distribution of players to understand the overall workforce composition in the dataset.

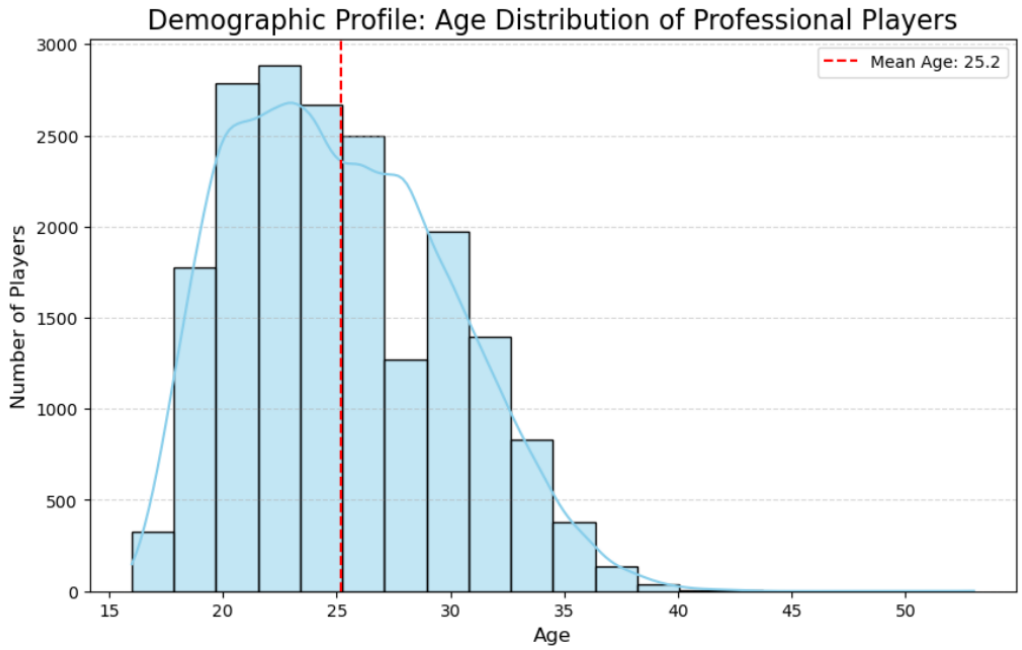
Plot: Histogram (or distribution plot) of the Age column.

- **X-axis:** Age
- **Y-axis:** Number of Players (Frequency)

SOLUTION:

In [27]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Create the Histogram with a Kernel Density Estimate (KDE) line
plt.figure(figsize=(10, 6))
sns.histplot(df['Age'], kde=True, color='skyblue', bins=20)

# 2. Add visual markers for context
mean_age = df['Age'].mean()
plt.axvline(mean_age, color='red', linestyle='--', label=f'Mean Age: {mean_age:.1f}')

# 3. Final touches
plt.title('Demographic Profile: Age Distribution of Professional Players', fontsize=16)
plt.xlabel('Age', fontsize=12)
plt.ylabel('Number of Players', fontsize=12)
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()
```

OUTPUT:



POWER BI DASHBOARD